MediCare — System Analysis & Design

DEPI Graduation Project

System Analysis & Design

1. Problem Statement & Objectives

Problem Statement

Appointment management in many clinics relies on manual or fragmented systems, leading to persistent operational challenges:

- **Double Booking:** Conflicting appointments occur because different parties rely on uncoordinated schedules.
- **Patient Access Barriers:** Patients face difficulty finding the right doctor based on specialty, location, and available time slots.
- **Disorganized Medical Records:** Medical history and prescriptions lack centralized organization and proper access control.
- **Lack of Automated Reminders:** Absence of automated appointment confirmations and status updates leads to high no-show rates.
- **Limited Operational Insights:** Clinic administration lacks key performance metrics to evaluate efficiency and workload.

System Overview: MediCare is a web-based clinic management and appointment booking system that connects patients with healthcare providers, enabling streamlined schedule coordination and secure medical record handling.

Key Stakeholders

Stakeholder	Role Type	Key Responsibilities & Description
Patient	Primary	Searches for doctors, books/cancels/reschedules appointments, and accesses medical history and prescriptions.
Doctor	Primary	Manages weekly schedule availability, reviews relevant patient info, and issues digital prescriptions.
Admin	Primary	Manages accounts, doctor profiles, system-wide appointments, system activity logs, and analytics

		dashboards.
Supervisor	Academic	Reviews system compliance against requirements and evaluates project deliverables.

Project Objectives

ID	Objective Description
O1	Enable patients to search for doctors filtered by specialty, location, and available time slots.
O2	Minimize schedule conflicts using server-side validation and atomic database transactions.
O3	Empower doctors to manage working hours, review appointment lists, and inspect patient history.
O4	Provide patients secure, read-only access to their authorized appointment logs and prescriptions.
O5	Deliver automated notifications for confirmations, reschedules, and cancellations.
O6	Equip administrators with tools for account management, doctor onboarding, and activity tracking.
O7	Improve overall clinic workflow coordination and provide metric insights into operational performance.

Project Scope Boundary

In-Scope (Supported Features)	Out-of-Scope (Excluded Features)
• Authentication & Role-Based Access Control	• Clinical Medical Diagnosis & Clinical Decision

(Patient, Doctor, Admin) • Doctor Profiles & Multi-Criteria Search (Specialty, Location, Availability) • Weekly Slot Availability & Schedule Management • Appointment Booking, Cancellation & Rescheduling • Server-Side Conflict Detection & Double-Booking Prevention • Patient & Doctor Dashboards • Authorized Medical Record & Prescription Viewing • Medical Document Upload (Optional) • System Notifications (In-App / Mock Provider) • Admin Account Management, Activity Logs & Analytics Dashboard	Support Systems • Emergency Care Management & Triage Handling • Integrated Financial Payments, Invoicing & Insurance Processing • Direct API Integration with External Hospital Information Systems (HIS) • Storage or Processing of Real Patient Identifiable Data (Mock Data Only)

1.1 Use Case Diagram & Descriptions

Actors Definition

Actor Name	Category	Description & Role Scope
Patient	Primary	Searches doctors, books/cancels/reschedules appointments, views records, and uploads documents.
Doctor	Primary	Configures weekly slot availability, manages appointments, views patient records, and issues prescriptions.
Admin	Primary	Oversees user accounts, manages doctor profiles, monitors system activity logs, and views analytics.
Notification Service	Secondary	Triggers system notifications (In-app / Mock / External Email) upon key event executions.

Use Case Summary Matrix

UC ID	Use Case Name	Primary / Secondary Actor(s)
UC-01	Register and Login	Patient, Doctor, Admin
UC-02	Search Doctors	Patient
UC-03	Book Appointment	Patient
UC-04	Cancel or Reschedule Appointment	Patient, Doctor
UC-05	View Appointment History	Patient
UC-06	View Records and Prescriptions	Patient
UC-07	Upload Document (Optional)	Patient, Doctor
UC-08	Manage Availability	Doctor
UC-09	Manage Appointments	Doctor
UC-10	View Patient Information	Doctor
UC-11	Write Prescription	Doctor
UC-12	Manage Accounts and Doctors	Admin
UC-13	View Analytics and Activity	Admin
UC-14	Send Notifications	Notification Service
UC-15	Check Availability and Detect Conflict	System (Included)

Detailed Core Use Case Specifications

UC-03: Book Appointment

Actor	Patient
Precondition	Patient is authenticated and doctor has published available slots.
Main Flow	1. Patient searches for a doctor (UC-02). 2. Patient selects doctor, date, and desired time slot. 3. Patient provides reason for visit. 4. Patient submits booking request. 5. System validates inputs and user authorization. 6. System performs re-check of slot availability inside a DB transaction (UC-15). 7. Appointment record is saved with status 'Pending' or 'Confirmed'. 8. System triggers notification to Patient and Doctor (UC-14).
Alternate Flows	A. Slot already taken: System rejects booking, shows conflict message, and suggests alternative open slots. B. Invalid input or past date: System presents inline error message.
Postcondition	Appointment is created in DB; slot is locked; duplicate booking for doctor at same slot is prevented.

UC-04: Cancel or Reschedule Appointment

Actor	Patient (Cancel/Reschedule), Doctor (Cancel)
Precondition	An active appointment exists in 'Pending' or 'Confirmed' status and is prior to execution date.
Main Flow	1. User opens appointment schedule. 2. User selects an existing appointment.

	3. User chooses either Cancel or Reschedule. 4. For rescheduling, user selects a new available slot (UC-15). 5. System updates appointment record and status. 6. System triggers notification (UC-14).
Alternate Flows	A. Reschedule slot unavailable: Request rejected; original booking remains intact. B. Policy window expired: Action rejected per cancellation rules.
Postcondition	Appointment status updated to 'Cancelled' or updated to new slot time; original slot freed.

UC-06: View Records and Prescriptions

Actor	Patient
Precondition	Patient is authenticated.
Main Flow	1. Patient navigates to Medical Records section. 2. System fetches records matching Patient Identity (Ownership Check). 3. Patient views list of consultations and prescriptions. 4. Patient opens specific prescription details.
Alternate Flows	A. No records found: System displays empty state message. B. Unauthorized access attempt: System denies access and logs unauthorized attempt in security audit log.
Postcondition	Data is presented securely without state mutation.

UC-08: Manage Availability

Actor	Doctor

Precondition	Doctor is authenticated.
Main Flow	1. Doctor opens Schedule Availability manager. 2. Doctor adds, updates, or deletes weekly recurring slots (Day, Start Time, End Time). 3. System validates time range (End Time > Start Time) and checks for overlapping intervals. 4. System persists updated schedule.
Alternate Flows	Modifying slot with active bookings: System prompts doctor to reschedule/cancel existing bookings first.
Postcondition	Updated availability slots are instantly reflected on public booking view.

UC-11: Write Prescription

Actor	Doctor
Precondition	Doctor is authenticated and owns the corresponding appointment record.
Main Flow	1. Doctor accesses appointment details after/during consultation. 2. Doctor inputs prescription items (Medication Name, Dosage, Frequency, Duration, Instructions). 3. Doctor saves prescription. 4. System links prescription to appointment and patient account. 5. Prescription becomes viewable in Patient's dashboard.
Alternate Flows	A. Appointment ownership check fails: Access denied. B. Missing required fields: Validation warnings displayed.
Postcondition	Prescription record stored and linked securely to patient record.

1.2 Functional & Non-Functional Requirements

Functional Requirements

ID	Requirement Description	Target Actor	Priority
FR-01	User registration and authentication with role assignment (Patient, Doctor, Admin).	All Users	**Must**
FR-02	Role-tailored personalized dashboard views for each user category.	All Users	**Must**
FR-03	Doctor profile setup including specialty, bio, clinic location, and consultation fees.	Doctor, Admin	**Must**
FR-04	Doctor search and multi-criteria filtering by specialty, location, and slot availability.	Patient	**Must**
FR-05	Configuration of weekly recurring availability slots.	Doctor	**Must**
FR-06	Booking of open time slots for selected doctors.	Patient	**Must**
FR-07	Appointment cancellation and slot rescheduling functionality.	Patient, Doctor	**Must**
FR-08	Server-side conflict detection and double-booking prevention logic.	System	**Must**
FR-09	Interactive schedule calendar visualization powered by FullCalendar.js.	Patient, Doctor	**Should**
FR-10	Appointment history view covering upcoming, completed, and cancelled visits.	Patient	**Must**
FR-11	Access to medical history for patients with established consultation relationships.	Doctor	**Must**
FR-12	Creation of digital prescriptions and secure patient	Doctor, Patient	**Must**

	access to authorized records.		
FR-13	Medical document and lab result attachment uploads.	Patient, Doctor	**Could**
FR-14	Automated appointment confirmation, rescheduling, and cancellation alerts.	System	**Should**
FR-15	Admin user account management, doctor profile verification, and system settings.	Admin	**Must**
FR-16	Admin analytical dashboards (Chart.js) and comprehensive system activity logs.	Admin	**Should**

Non-Functional Requirements

ID	Category	Requirement Description	Target Measurement Metric
NFR-01	Performance	Fast query response time for doctor search and slot booking.	Median response time ≤ 2.0 seconds under standard loads.
NFR-02	Security	Robust Authentication, RBAC, ownership checks, secure sessions, password hashing.	Zero high-risk access control vulnerabilities in security testing.
NFR-03	Data Integrity	Double-booking prevention guaranteed via database transaction isolation.	0 accepted overlapping appointment conflicts during concurrency tests.
NFR-04	Availability	High operational uptime during evaluation and presentation windows.	≥ 99.0% system uptime maintained across evaluation phase.
NFR-05	Usability	Responsive, intuitive UX design built with Bootstrap 5 framework.	≥ 80% of test users complete key tasks without guidance.

NFR-06	Privacy	Compliance with patient data privacy; synthetic mock data strictly enforced.	100% synthetic data verified prior to repository release.
NFR-07	Maintainability	Clean Layered MVC architecture, Git source control, and documented APIs.	All code integrated via reviewed GitHub Pull Requests.
NFR-08	Testability	Comprehensive unit and integration test coverage for core workflows.	≥ 95% pass rate on test suites with zero critical open bugs.
NFR-09	Notification Reliability	Decoupled notification architecture with fallback in-app/mock providers.	≥ 95% successful delivery rate in test environment.
NFR-10	Auditability	Comprehensive logging of administrative and sensitive data operations.	Persistent activity log entries accessible via Admin panel.

1.3 Software Architecture

Architecture Pattern & Rationale

MediCare is engineered using a Layered MVC Monolith Architecture built on ASP.NET Core MVC. A monolithic architectural style was deliberately selected over Microservices due to the project size, team composition, and strict eight-week timeline. This structure eliminates distributed system complexity while maintaining clean separation of concerns, enabling effective parallel development across frontend and backend modules.

Layered Structure & Technical Stack

Layer	Key Responsibilities	Technologies & Frameworks
Presentation Layer	UI rendering, client-side validation, interactive calendars, and analytical chart displays.	Razor Views, Bootstrap 5, JavaScript, FullCalendar.js, Chart.js

Middleware Layer	Global authentication, role/policy authorization, and centralized exception handling.	ASP.NET Core Identity, Authorization Policies, Custom Middleware
Controllers Layer	Request handling, HTTP orchestration, ViewModel mapping, and service invocation.	ASP.NET Core MVC Controllers
Service Layer	Encapsulation of core business logic: availability checks, conflict prevention, notification dispatch.	C# Services, Dependency Injection
Data Access Layer	Data persistence abstraction, ORM mappings, repository pattern, and transaction management.	Entity Framework Core, Repositories, DbContext
Database Layer	Relational data storage for users, doctor schedules, appointments, prescriptions, and logs.	Microsoft SQL Server
External Services	Notification dispatch abstraction (In-App, Mock, or Email delivery providers).	In-App System, Mock Provider, Email Interface

Key Architectural Decisions (ADR)

Architectural Decision	Design Rationale & Benefits
Layered Monolith with MVC Pattern	Fits team size and project timeframe perfectly; simplifies deployment and continuous integration while enforcing clean modularity.
Business Logic in Service Layer	Keeps MVC Controllers lean (thin controllers), enhances reusability, and facilitates isolated unit testing of validation rules.
Transaction Re-checks & Unique Constraints	Prevents race conditions and double-booking at both application logic and database engine levels.

Strict Role & Ownership Verification	Ensures patients and doctors only access data they own, guaranteeing HIPAA-aligned authorization boundaries.
Interface-Based Notification Architecture	Allows seamless switching between in-app notifications, mock providers, and real SMTP services without altering business code.